CO4-AT4: PART II

Design and Develop Model - Sampling-Based Planning & Data Generation

Name	Guggilla Venkatasai	Register No.	192425388
Programme / Section	Reinforcement Learning	Course Code	MLA0305
Assessment Tool	Model-Based Assessment	AT No.	4 - Part II
Assessment Type	Sampling-Based Planning & Data Generation	Bloom Level	L3 / L4
Faculty	Dr. Vincent Gnanaraj T	Date	19/08/2026

Aim

To design and develop a model for sampling-based path planning and data generation using Q-Learning in a grid environment containing obstacles, and to visualize the learned path from the start state to the goal state.

Method

A 10 x 10 grid is represented as an environment. The agent uses four movement actions, receives rewards or penalties based on the next state, and updates a Q-table over 5000 training episodes. After training, the greedy policy is used to extract and plot the learned path.

Algorithm Parameters

Parameter	Value
Grid size (N)	10 x 10
Learning rate (alpha)	0.1
Discount factor (gamma)	0.9
Exploration rate (epsilon)	0.2
Training episodes	5000
Maximum steps / episode	200
Actions	Up / Down / Left / Right

Python Implementation

```
import numpy as np import
matplotlib.pyplot as plt
import random

N = 10 start = (0, 0) goal = (9, 9) obstacles =
 [(3, 3), (3, 4), (4, 3), (6, 6), (7, 6)]

actions = [(-1, 0), (1, 0), (0, -1), (0,
1)] Q = np.zeros((N, N, 4))

alpha = 0.1
gamma = 0.9
epsilon = 0.2

# Q-learning training for
episode in range(5000):
```

```
 state = start
 for step in range(200): if
random.random() < epsilon: action =
random.randint(0, 3) else:
action = np.argmax(Q[state[0], state[1]])
 dx, dy =
actions[action] x =
 state[0] + dx y =
 state[1] + dy
 if x < 0 or x >= N or y < 0 or y >= N:
 reward = -10
next_state = state elif
 (x, y) in obstacles:
reward = -10
next_state = state elif
 (x, y) == goal:
reward = 100
next_state = (x, y)
else:
 reward = -1
next_state = (x, y)
 old_value = Q[state[0], state[1], action]
best_next = np.max(Q[next_state[0],
next_state[1]])
 Q[state[0], state[1], action] = ( old_value +  alpha * (reward
+ gamma * best_next - old_value) )  state = next_state
 if state == goal:
break
# Extract the learned
path state = start path =
 [state]
for _ in range(100):
 action = np.argmax(Q[state[0], state[1]])
dx, dy = actions[action] next_state =
 (state[0] + dx, state[1] + dy)
 if next_state == state or next_state in obstacles:
break
 path.append(next_state)
 state = next_state
 if state == goal:
break
# Visualize the path
plt.figure(figsize=(7, 7))
for x, y in obstacles: plt.scatter(y, N -
1 - x, s=700, marker='s')
px = [p[1] for p in path] py = [N
- 1 - p[0] for p in path]
plt.plot(px, py, 'o-',
linewidth=3)
plt.scatter(start[1], N - 1 - start[0], s=150,
label='Start')  plt.scatter(goal[1], N - 1 - goal[0],
s=150, label='Goal')
plt.xlim(-0.5, N - 0.5) plt.ylim(-0.5, N - 0.5)
plt.xticks(range(N)) plt.yticks(range(N))
plt.grid(True) plt.title('RL-Based Path Planning
using Q-Learning') plt.xlabel('X')
plt.ylabel('Y')  plt.legend() plt.show()
```

Output - RL-Based Path Planning using Q-Learning

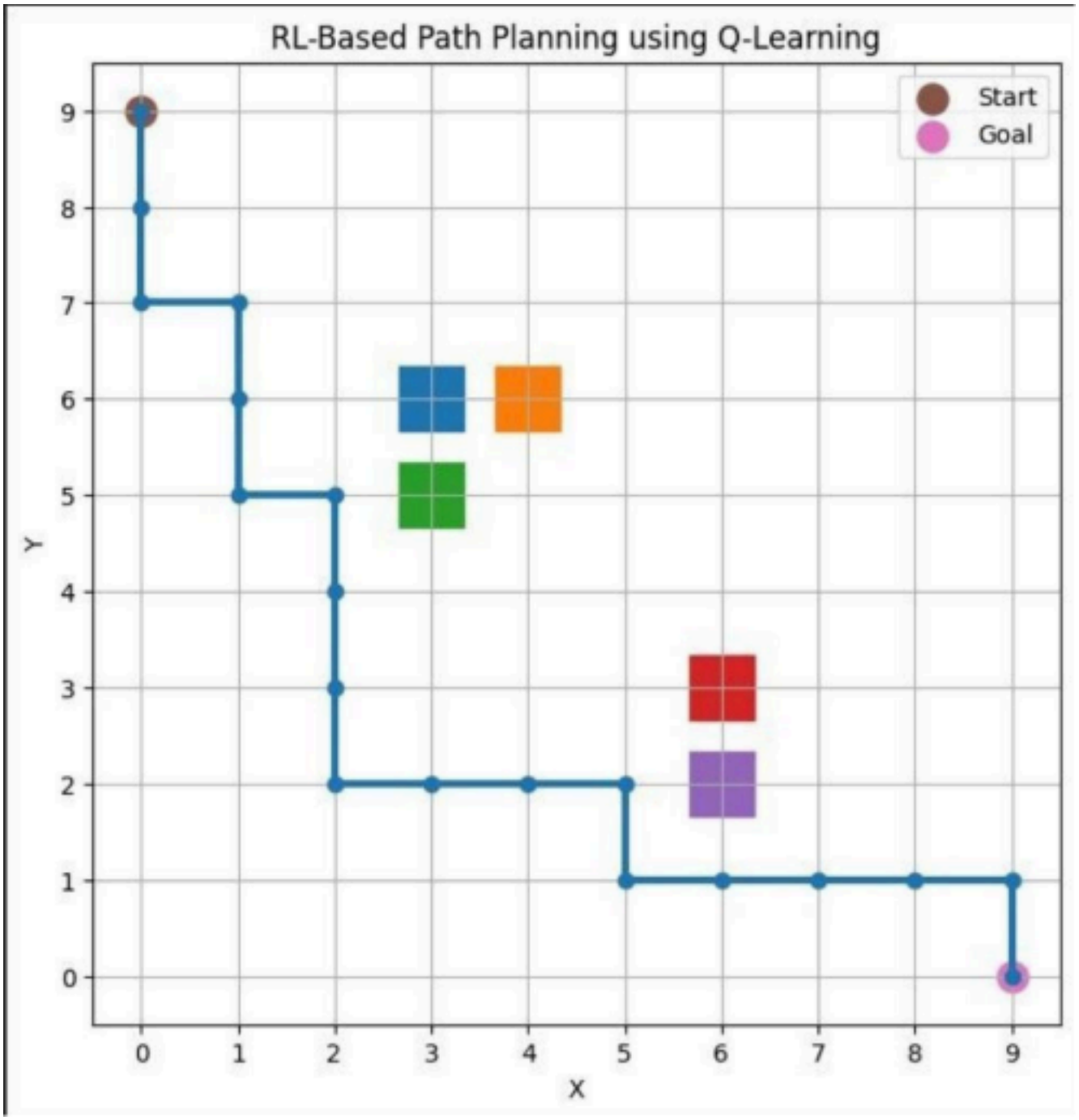

Figure 1. Learned path generated by the Q-Learning agent in the grid environment. The plot shows the start state, goal state, obstacle cells, and the resulting path.

Result

The Q-Learning agent learns a path-planning policy through repeated interaction with the grid environment. The final visualization presents the learned route while avoiding the defined obstacle cells and moving toward the goal state.

Conclusion

The model demonstrates how reinforcement learning can be used for model-based path planning and trajectory generation in a discrete grid environment. The Q-table captures the learned action values and enables the agent to select a path using the learned policy.